

Глава 5: Следование вдоль стен и навигация

Учимся выравниваться вдоль стен и ориентироваться в комнате

В Главе 4 ты научился останавливаться перед стеной. Теперь научись выравниваться параллельно стенам и перемещаться по комнате.

Что ты изучишь

1. Измерение параллельности роботу к стене (метод двух точек)
2. Выравнивание параллельно стене поворотом
3. Движение с поддержанием выравнивания
4. Использование `check_wall()` для простого выравнивания
5. Использование `get_clearance()` для безопасного обнаружения препятствий
6. Навигация по квадратной комнате

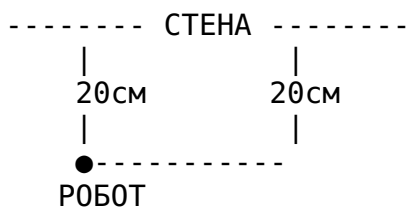
Проверка выравнивания по двум точкам

Чтобы узнать, параллелен ли ты стене, измерь расстояние под двумя углами.

Как это работает

Если измерить расстояние под двумя близкими углами и они одинаковые, ты параллелен!

Робот параллелен стене:



Робот под углом к стене:



Идея: Измерь под углами 80° и 100° (разница 20°). Если расстояния разные, робот под углом.

Простая проверка выравнивания

```
import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)
```

```

# Измеряем две точки справа
d1 = robot.lidar.get_distance_at_angle(80)
d2 = robot.lidar.get_distance_at_angle(100)

if d1 and d2:
    diff = d1 - d2
    print(f"Расстояние на 80°: {d1:.2f}м")
    print(f"Расстояние на 100°: {d2:.2f}м")
    print(f"Разница: {diff:.2f}м")

    if abs(diff) < 0.02: # Меньше 2см разницы
        print("Параллельно стене!")
    else:
        print("Не параллельно - нужно повернуть")

robot.stop()

```

Как читать разницу: - diff > 0: Перед ближе к стене, нужно повернуть ЧС (по часовой)
 - diff < 0: Зад ближе к стене, нужно повернуть ПЧС (против часовой) - diff ≈ 0: Параллельно!

Выравнивание к стене

Теперь повернемся, пока не станем параллельны.

Выравнивание поворотом

```

import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

# Поворачиваемся, пока не станем параллельны правой стене
while True:
    d1 = robot.lidar.get_distance_at_angle(80)
    d2 = robot.lidar.get_distance_at_angle(100)

    if d1 and d2:
        diff = d1 - d2

        if abs(diff) < 0.02: # Параллельно!
            break

    # Поворачиваемся для выравнивания
    if diff > 0:
        robot.drive.move(vtheta=-0.2) # Поворот ЧС

```

```

        else:
            robot.drive.move(vtheta=0.2)    # Поворот ПЧС

    time.sleep(0.1)

robot.drive.halt()
robot.stop()

```

Как это работает: 1. Измеряем две точки 2. Вычисляем разницу 3. Если не параллельно, поворачиваем в нужную сторону 4. Повторяем, пока не станем параллельны

Упражнение 5.1: Выравнивание к передней стене

Напиши код для выравнивания параллельно передней стене (используй углы 350° и 10°).

Решение

```

import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

while True:
    d1 = robot.lidar.get_distance_at_angle(350)
    d2 = robot.lidar.get_distance_at_angle(10)

    if d1 and d2:
        diff = d1 - d2

        if abs(diff) < 0.02:
            break

        if diff > 0:
            robot.drive.move(vtheta=-0.2)
        else:
            robot.drive.move(vtheta=0.2)

    time.sleep(0.1)

robot.drive.halt()
robot.stop()

```

Движение с выравниванием

Теперь едем вперед, сохраняя параллельность стене.

Цикл движения и выравнивания

```
import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

# Едем вперед 3 секунды, сохраняя выравнивание с правой стеной
start_time = time.time()

while time.time() - start_time < 3.0:
    # Проверяем выравнивание
    d1 = robot.lidar.get_distance_at_angle(80)
    d2 = robot.lidar.get_distance_at_angle(100)

    if d1 and d2:
        diff = d1 - d2

        # Корректируем поворот для сохранения выравнивания
        if abs(diff) < 0.02:
            vtheta = 0 # Параллельно, без поворота
        elif diff > 0:
            vtheta = -0.1 # Маленький поворот ЧС
        else:
            vtheta = 0.1 # Маленький поворот ПЧС

        # Едем вперед с коррекцией выравнивания
        robot.drive.move(vx=0.1, vtheta=vtheta)

    time.sleep(0.1)

robot.drive.halt()
robot.stop()
```

Как это работает: 1. Измеряем выравнивание во время движения 2. Если не параллельно, добавляем небольшой поворот 3. Продолжаем ехать вперед с коррекцией поворота 4. Результат: робот едет прямо, оставаясь параллельным!

Упражнение 5.2: Движение вбок с выравниванием

Измени код, чтобы двигаться влево, сохраняя выравнивание с передней стеной.

Решение

```
import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
```

```

robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

start_time = time.time()

while time.time() - start_time < 3.0:
    d1 = robot.lidar.get_distance_at_angle(350)
    d2 = robot.lidar.get_distance_at_angle(10)

    if d1 and d2:
        diff = d1 - d2

        if abs(diff) < 0.02:
            vtheta = 0
        elif diff > 0:
            vtheta = -0.1
        else:
            vtheta = 0.1

        robot.drive.move(vy=0.1, vtheta=vtheta)

    time.sleep(0.1)

robot.drive.halt()
robot.stop()

```

Использование check_wall()

Вычислять выравнивание вручную работает, но есть способ проще!

Функция check_wall()

```
distance, angle_deg, quality = robot.lidar.check_wall(90)
```

Возвращает: - distance: Перпендикулярное расстояние до стены (метры) - angle_deg: Насколько нужно повернуть, чтобы быть параллельным (градусы) - Положительное = повернуть ЧС для выравнивания - Отрицательное = повернуть ПЧС для выравнивания - quality: Насколько хорошо измерение (0-1, выше = лучше)

Простое выравнивание с check_wall()

```

import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
robot.start()

```

```

time.sleep(3)

# Выравниваемся к правой стене
while True:
    distance, angle_deg, quality = robot.lidar.check_wall(90)

    if angle_deg is not None:
        if abs(angle_deg) < 2.0: # В пределах 2 градусов
            break

    # Поворачиваемся пропорционально ошибке
    vtheta = 0.02 * angle_deg # Положительный угол = поворот ЧС
    robot.drive.move(vtheta=vtheta)

    time.sleep(0.1)

robot.drive.halt()
robot.stop()

```

Намного проще! Не нужно измерять две точки или вычислять разницу.

Упражнение 5.3: Выравнивание к передней стене с check_wall()

Используй check_wall() для выравнивания к передней стене (угол 0).

Решение

```

import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

while True:
    distance, angle_deg, quality = robot.lidar.check_wall(0)

    if angle_deg is not None:
        if abs(angle_deg) < 2.0:
            break

    vtheta = 0.02 * angle_deg
    robot.drive.move(vtheta=vtheta)

    time.sleep(0.1)

robot.drive.halt()
robot.stop()

```

Использование get_clearance()

При проверке расстояния до стены, get_clearance() безопаснее, чем get_distance_at_angle().

Почему get_clearance() лучше

```
# get_distance_at_angle() - измеряет ОДИН угол
distance = robot.lidar.get_distance_at_angle(0)
```

```
# get_clearance() - смотрит на ДИАПАЗОН углов
distance = robot.lidar.get_clearance(angle=0, robot_width=0.22, angular_range=3.14)
```

Преимущества: - Проверяет несколько углов (безопаснее - не пропустит препятствия) -
Учитывает ширину робота - Возвращает ближайшее препятствие в области

Приближение к стене с get_clearance()

```
import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

# Движемся вперед до 20см от стены
robot.drive.move(vx=0.1)

while True:
    clearance = robot.lidar.get_clearance(angle=0, robot_width=0.22)

    if clearance and clearance < 0.20:
        break

    time.sleep(0.05)

robot.drive.halt()
robot.stop()
```

Следование вдоль стены

Теперь объединим всё: приближение к стене, выравнивание, затем движение вдоль неё.

Приближение, выравнивание и следование

```
import time
from evabot import Robot, MecanumDrive, RPLidarC1
```

```

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

# Шаг 1: Приближаемся к передней стене
robot.drive.move(vx=0.1)
while True:
    clearance = robot.lidar.get_clearance(0, 0.22)
    if clearance and clearance < 0.20:
        break
    time.sleep(0.05)
robot.drive.halt()
time.sleep(0.5)

# Шаг 2: Выравниваемся к передней стене
while True:
    distance, angle_deg, quality = robot.lidar.check_wall(0)
    if angle_deg and abs(angle_deg) < 2.0:
        break
    if angle_deg:
        robot.drive.move(vtheta=0.02 * angle_deg)
    time.sleep(0.1)
robot.drive.halt()
time.sleep(0.5)

# Шаг 3: Двигаемся влево, сохраняя выравнивание
start_time = time.time()
while time.time() - start_time < 3.0:
    # Проверяем левую стену
    left_clear = robot.lidar.get_clearance(270, 0.22)
    if left_clear and left_clear < 0.20:
        break

    # Поддерживаем выравнивание с передней стеной
    distance, angle_deg, quality = robot.lidar.check_wall(0)
    vtheta = 0.02 * angle_deg if angle_deg else 0

    robot.drive.move(vy=0.1, vtheta=vtheta)
    time.sleep(0.1)

robot.drive.halt()
robot.stop()

```

Что это делает: 1. Двигается вперед к передней стене 2. Поворачивается для выравнивания параллельно 3. Двигается влево, оставаясь параллельным передней стене 4. Останавливается при достижении левой стены

Навигация по квадрату

Теперь объединим квадратную комнату!

Навигация вокруг комнаты

```
import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

# Стены: 0=перед, 90=право, 180=зад, 270=лево
walls = [
    (0, 'vx', 0.1),      # Приближение к передней стене
    (270, 'vy', 0.1),    # Движение к левой стене (следуя передней)
    (180, 'vx', -0.1),   # Движение к задней стене (следуя левой)
    (90, 'vy', -0.1),    # Движение к правой стене (следуя задней)
]

for i, (target_wall, move_dir, speed) in enumerate(walls):
    # Определяем, какой стене следовать
    if i == 0:
        follow_wall = None # Первое движение, просто приближаемся
    else:
        follow_wall = walls[i-1][0] # Следуем предыдущей стене

    print(f"Движемся к стене {target_wall}")

    # Движемся, пока не достигнем целевой стены
    while True:
        clearance = robot.lidar.get_clearance(target_wall, 0.22)
        if clearance and clearance < 0.20:
            break

        # Поддерживаем выравнивание с follow_wall, если есть
        vtheta = 0
        if follow_wall is not None:
            dist, angle_deg, quality = robot.lidar.check_wall(follow_wall)
            if angle_deg:
                vtheta = 0.02 * angle_deg

        # Движемся к цели
        if move_dir == 'vx':
            robot.drive.move(vx=speed, vtheta=vtheta)
        else:
            robot.drive.move(vy=speed, vtheta=vtheta)

        time.sleep(0.1)

    robot.drive.halt()
    time.sleep(0.5)
```

```

# Выравниваемся к целевой стене
print(f"Выравнивание к стене {target_wall}")
while True:
    dist, angle_deg, quality = robot.lidar.check_wall(target_wall)
    if angle_deg and abs(angle_deg) < 2.0:
        break
    if angle_deg:
        robot.drive.move(vtheta=0.02 * angle_deg)
    time.sleep(0.1)

robot.drive.halt()
time.sleep(0.5)

print("Квадрат завершен!")
robot.stop()

```

Что это делает: 1. Двигается к передней стене → выравнивается 2. Двигается влево (следуя передней стене) → достигает левой стены → выравнивается 3. Двигается назад (следуя левой стене) → достигает задней стены → выравнивается 4. Двигается вправо (следуя задней стене) → достигает правой стены → выравнивается 5. Робот объехал периметр!

Упражнение 5.4: Возврат к старту

После завершения квадрата, добавь еще одно движение для возврата к передней стене (замыкание цикла).

Решение

Добавь это после цикла выше:

```

print("Возвращаемся к передней стене")

# Движемся вперед, следуя правой стене
while True:
    clearance = robot.lidar.get_clearance(0, 0.22)
    if clearance and clearance < 0.20:
        break

    dist, angle_deg, quality = robot.lidar.check_wall(90)
    vtheta = 0.02 * angle_deg if angle_deg else 0

    robot.drive.move(vx=0.1, vtheta=vtheta)
    time.sleep(0.1)

robot.drive.halt()
print("Вернулись к старту!")
robot.stop()

```

Резюме

Ты изучил:

□ Ручная проверка выравнивания по двум точкам □ Поворот для выравнивания параллельно стенам □ Движение с поддержанием выравнивания □ Использование `check_wall()` для простого выравнивания - Возвращает расстояние, ошибку угла и качество - Положительный угол = поворот ЧС, отрицательный = поворот ПЧС □ Использование `get_clearance()` для безопасного обнаружения препятствий - Проверяет диапазон углов, а не одну точку - Учитывает ширину робота □ Следование вдоль стены (приближение → выравнивание → следование) □ Навигация по квадрату вокруг комнаты

Ключевой шаблон для следования вдоль стены:

```
# Движемся к стене
robot.drive.move(vx=0.1, vtheta=коррекция_выравнивания)

while True:
    # Проверяем расстояние до целевой стены
    clearance = robot.lidar.get_clearance(целевой_угол, 0.22)
    if clearance < 0.20:
        break

    # Поддерживаем выравнивание с follow_wall
    dist, angle_deg, quality = robot.lidar.check_wall(угол_следования)
    vtheta = 0.02 * angle_deg if angle_deg else 0

    robot.drive.move(vx=скорость, vtheta=vtheta)
    time.sleep(0.1)

robot.drive.halt()
```

Что дальше?

Глава 6 научит тебя решению лабиринтов: - Обнаружение углов и проходов - Принятие навигационных решений - Следование стратегии левой или правой стены - Поиск выхода!

Готов к лабиринту? □